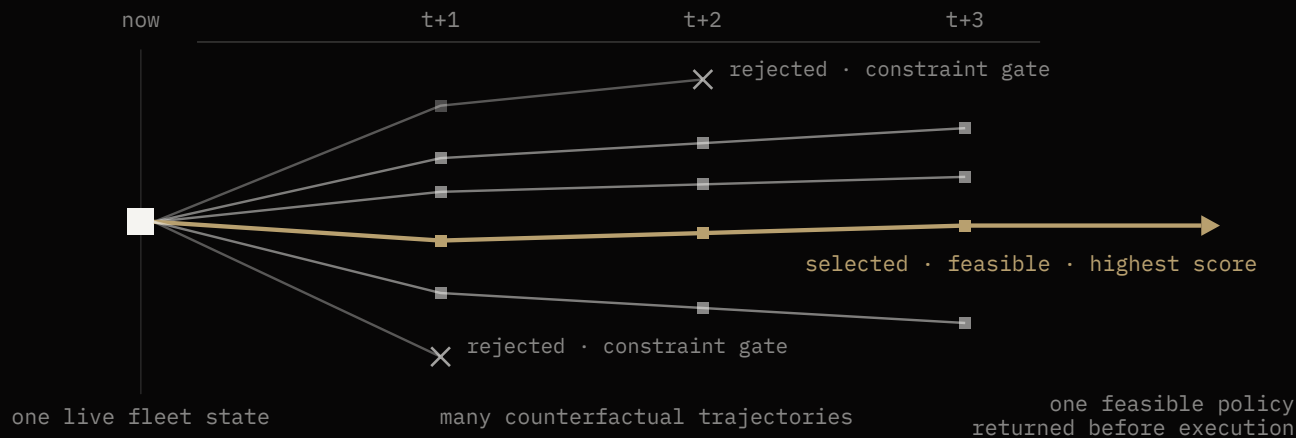


Choose the best fleet trajectory, not just the best next action.

One live fleet state. Many counterfactual futures. One safe trajectory selected before execution.



Kubernetes, Slurm, serving schedulers, autoscalers, and fleet-management systems already own specialized control and execution decisions. Aurelius adds a pre-execution planning layer above them.

Before a decision is committed, Aurelius forks the current fleet state into competing simulated trajectories. Each trajectory applies a different coupled policy across the controls implemented for that workload, including batching, routing, placement, capacity, prewarming, admission, migration, precision, speculative decoding, and GPU clock policy.

The fleet state is then advanced through the planning horizon. Each trajectory changes queue progression, capacity use, placement and topology pressure, replica warm state, SLA outcomes, and modeled infrastructure cost.

Trajectories that violate configured SLA, capacity, power, residency, placement, or policy constraints are rejected before ranking. The highest-scoring feasible policy is returned to the existing control plane, which remains authoritative and executes through its normal interfaces.

8.24×

mean SLA-safe goodput per modeled infrastructure dollar versus a constructed production-informed replay baseline

84.5%

1,549,249

3 of 3

fewer GPU-hours under identical input request sets

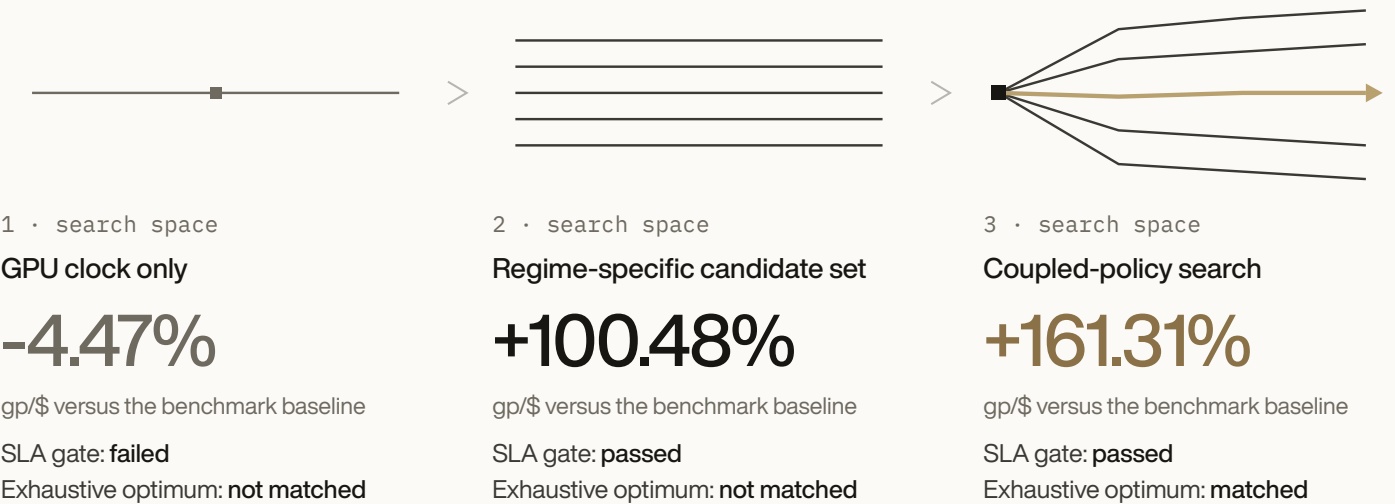
requests across three selected uncapped high-load windows

windows with lower SLA violation rates

Simulated, load-dependent evidence from selected public-trace windows. Not a production result and not an estimate of performance against a particular fleet or internal scheduler.

The optimizer cannot choose a future it never represents.

The frozen V1 ablation held the forecast, simulator, objective, cost model, and constraint gates constant. Only the set of policies available to the planner changed.



Within the frozen V1 primary window, the winning permitted policy combined lower precision, more aggressive batching, and a higher GPU clock.

A clock-only search never represented that policy. Tuned one control at a time, the required joint policy never entered the candidate set.

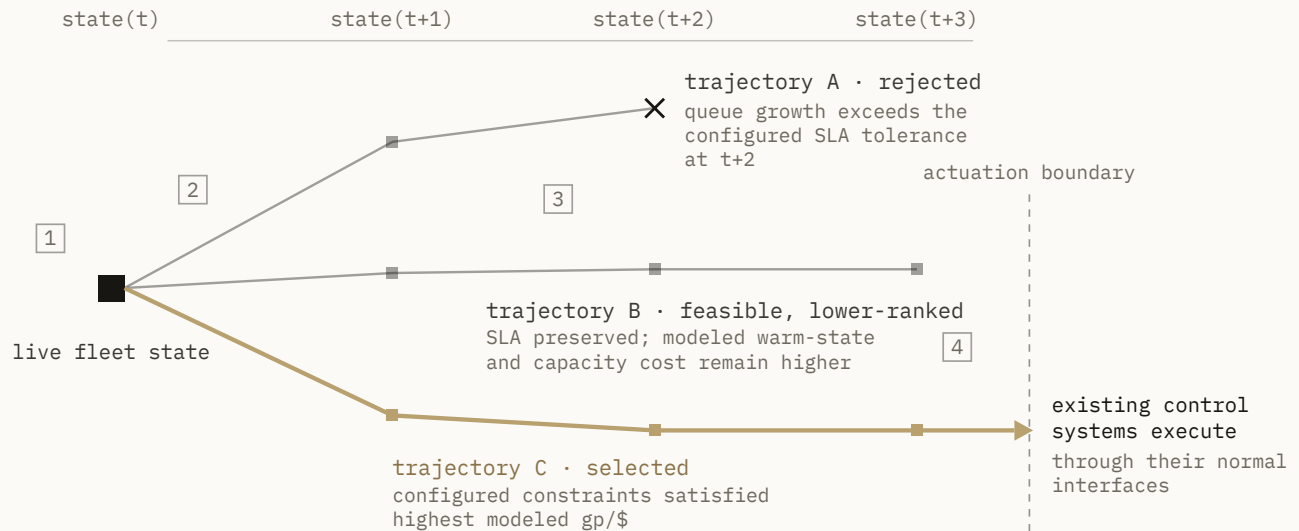
Adding the regime-specific candidate set turned a regression into a large positive result. Allowing coupled search across the implemented policy dimensions increased the result again and matched exhaustive enumeration.

The forecast, simulator, objective, cost model, and constraint gates were unchanged.

The value was not hidden in a better score. It was hidden in a future the previous search never considered.

Frozen Benchmark V1, primary market window. Different load and comparison policy from the uncapped replay. Where enumeration was tractable, the coupled search matched the exhaustive feasible optimum at approximately 40 candidate evaluations per decision. This validates search inside those enumerated spaces; it does not validate the simulator or establish production savings.

One fleet. Many futures. Simulated before execution.



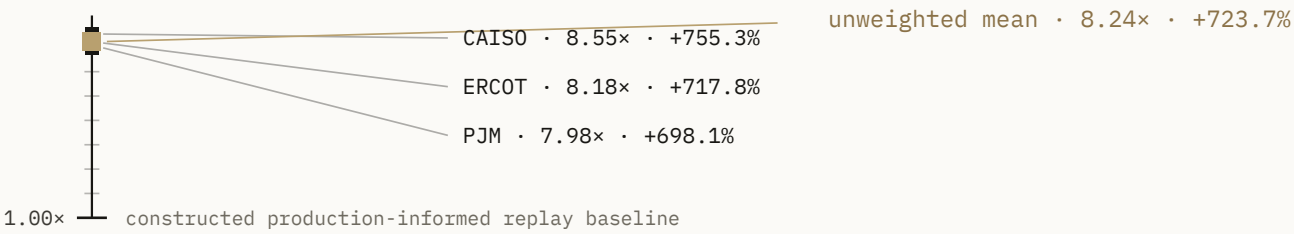
Live fleet state: queue · capacity · placement · topology · replica state · configured constraints. Each candidate runs against isolated simulated state. Candidate evaluation never writes to the live fleet.

- 1 **Mirror the fleet.**
A persistent state-transition model carries queue state, capacity, placement and locality, replica warm and cold state, migrations in flight, configured constraints, and the modeled cost basis across control periods.
- 2 **Generate coupled policies.**
A regime-specific generator proposes a bounded set of policies based on the workload, operating conditions, implemented controls, and configured constraints. Previously validated candidates are retained as controls.
- 3 **Advance each trajectory.**
Every candidate is simulated independently over the planning horizon. Forecasts use only information available at the decision timestamp. Future arrivals, future prices, and future outcomes are not supplied to the planner.
- 4 **Reject infeasible trajectories and return the winner.**
Hard constraints reject infeasible trajectories before economic ranking. The highest-scoring feasible policy is returned as a recommendation. When forecast confidence falls below the configured threshold, Aurelius returns the configured deterministic fallback policy.

Aurelius sits above Kubernetes, Slurm, serving schedulers, routing systems, autoscalers, and fleet-management systems. It evaluates how their decisions interact across a shared planning horizon. It does not replace them, write cluster state, or bypass operator-configured constraints.

The proposed historical replay requires scheduler and fleet metadata. It does not require customer identity, request payloads, model outputs, or proprietary weights.

A signal too large to ignore and too simulated to trust without operator data.



1,549,249 total input requests · 84.5% fewer total GPU-hours · lower SLA violation rates in all three windows · both policies received the identical input request sets.

The comparison policy is a deterministic public-replay baseline assembled from established serving and cluster-control mechanisms, including continuous batching, deadline-aware ordering and admission, prefix-cache-aware routing, topology-aware placement, and backlog-driven capacity control. It is stronger than the naive SLA-aware reference, but it is not an operator's internal scheduler and does not reproduce a particular fleet's adaptive logic, internal signals, or cost model.

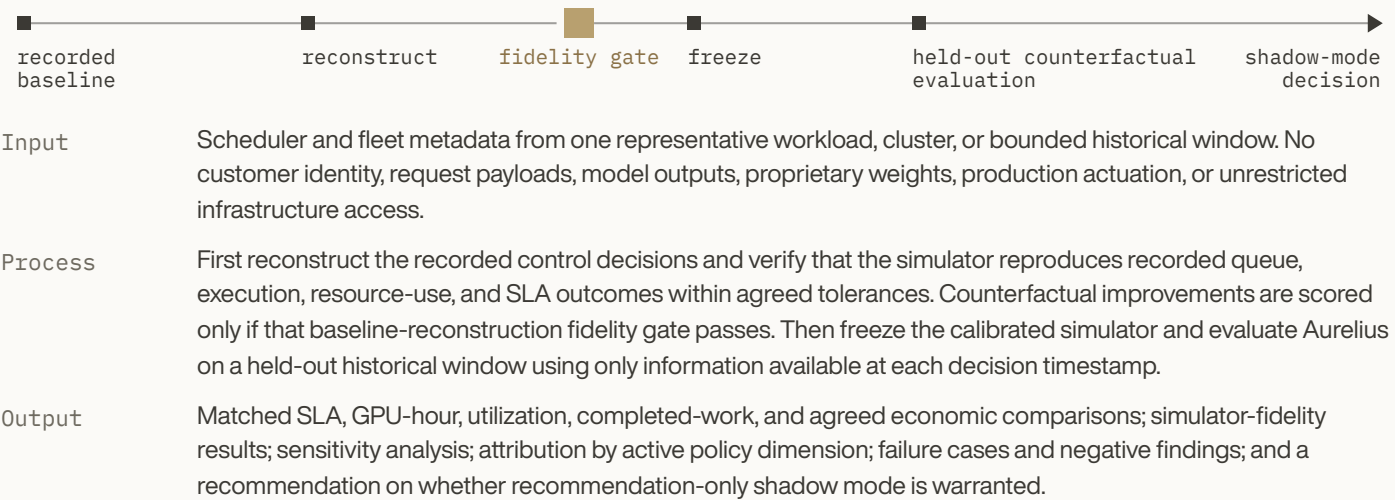
What this establishes

Under the selected uncapped high-load replay conditions, coupled predictive planning found feasible trajectories with materially better modeled economics and lower SLA violation rates than the constructed baseline.

What this does not establish

Expected performance against a particular fleet, internal scheduler, workload distribution, or operator cost model. The result is simulated and load-dependent. Its magnitude is materially affected by the benchmark's batching, precision, service-time, and modeled cost assumptions.

Test one bounded historical window.



Requested next step

A short scoping session with one engineering owner to select a bounded historical window, validate the minimum metadata schema, and agree on replay acceptance criteria.

If Aurelius cannot reproduce baseline behavior and improve the pre-registered primary metric on held-out telemetry, the evaluation stops. If it can, the next step is recommendation-only shadow mode.

Beckett Zahedi Founder · Aurelius beckett@runaurelius.com runaurelius.com

Complete methodology, references, benchmark tables, limitations, and mechanism audits are in the supporting full technical report.